# Final Report - Computer Networks Project

## Part 1: Packet Encapsulation and Wireshark Capture

### 1) How the CSV file was created:

For Part 1, I created a CSV file that represents messages for a simple chat-style protocol. Each row describes one message event including the protocol type, source/destination ports, message content, and timestamp. The CSV was generated using ChatGPT:

- Clients identifying themselves (HELLO)
- Starting private chats (CHAT)
- Exchanging messages (MSG)
- Disconnecting (BYE)

### 2) Description and explanation of the encapsulation process:

The CSV messages start as application-layer data (plain text strings such as HELLO alice).

In the provided Jupyter notebook, each message is converted into a packet by manual encapsulation (packet crafting), meaning the notebook builds the protocol headers itself and sends the packet on the loopback interface.

- Application Layer
    - The message text is created as a plain string (example: HELLO alice, MSG hi bob!).
    - This text becomes the payload that we want to transmit.
    - Transport Layer (TCP)

- Transport Layer (TCP)
  The notebook constructs a TCP header that includes fields such as:
    - Source port (random high port, e.g., 52962)
    - Destination port (12345)
    - Sequence/Acknowledgment numbers
    - TCP flags (e.g., SYN, ACK, PSH, RST)
    - Checksum

The message text is inserted as the TCP payload.

- Network Layer (IP)
  The notebook encapsulates the TCP segment inside an IP packet and sets:
  - Source IP = 127.0.0.1
  - Destination IP = 127.0.0.1
  - Protocol field = TCP

- Link Layer (Loopback)
  The final packet is transmitted through the loopback interface and captured in Wireshark.

- What was observed in Wireshark:
  Although packets were successfully created and captured, the capture did not show a full TCP connection lifecycle (3-way handshake + normal data transfer + clean termination).
  Instead, many packets were marked with RST (Reset) and retransmissions.
  This indicates that the operating system rejected the packets, therefore resetting the connection.
  Despite this, the capture still demonstrates the encapsulation concept: the application message is visible inside the TCP payload, inside an IP packet, transmitted over the loopback interface.

## 3) Description of the capture process

To capture the traffic:

1. Opened Wireshark
2. Selected the active interface (loopback for localhost testing)
3. Started packet capture
4. Ran the Jupyter notebook to send messages based on the CSV file
5. Stopped the capture after the messages were sent
6. Saved the capture file (.pcap / .pcapng)

## 4) Wireshark traffic analysis

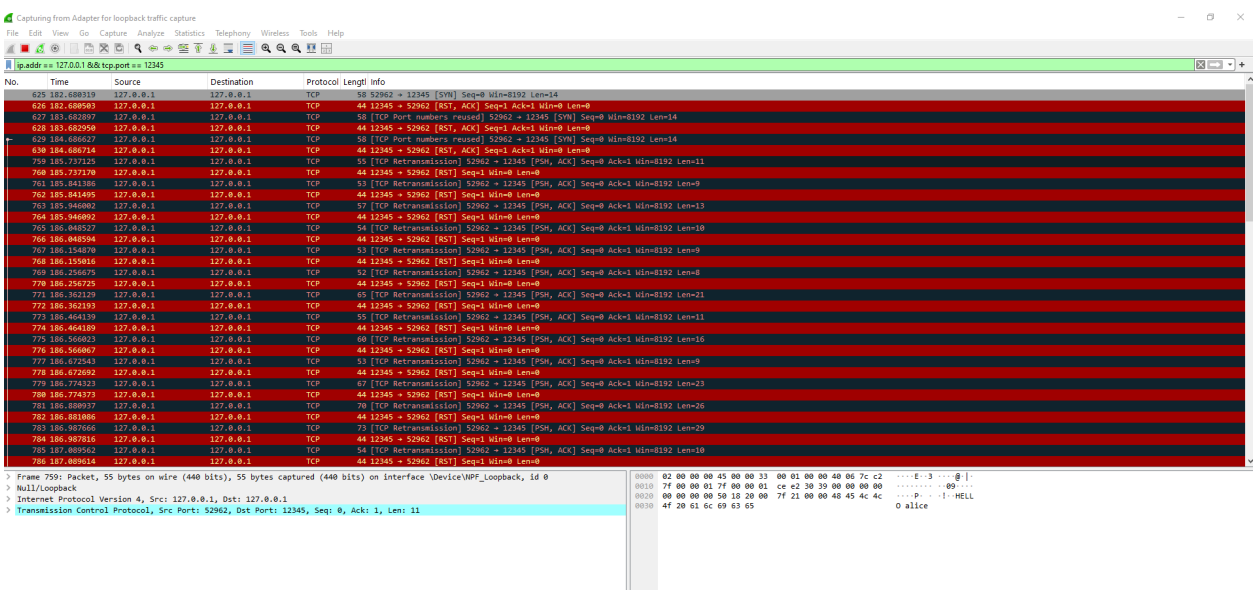
In Wireshark, I verified that the packets contain:

- TCP layer
  - Source port matches the configured/generated source port
  - Destination port matches the server port (12345)
- IP layer
  - Source IP = 127.0.0.1
  - Destination IP = 127.0.0.1
- Payload
  - Contains the same message text as the CSV row (example: HELLO alice, MSG hi bob!)

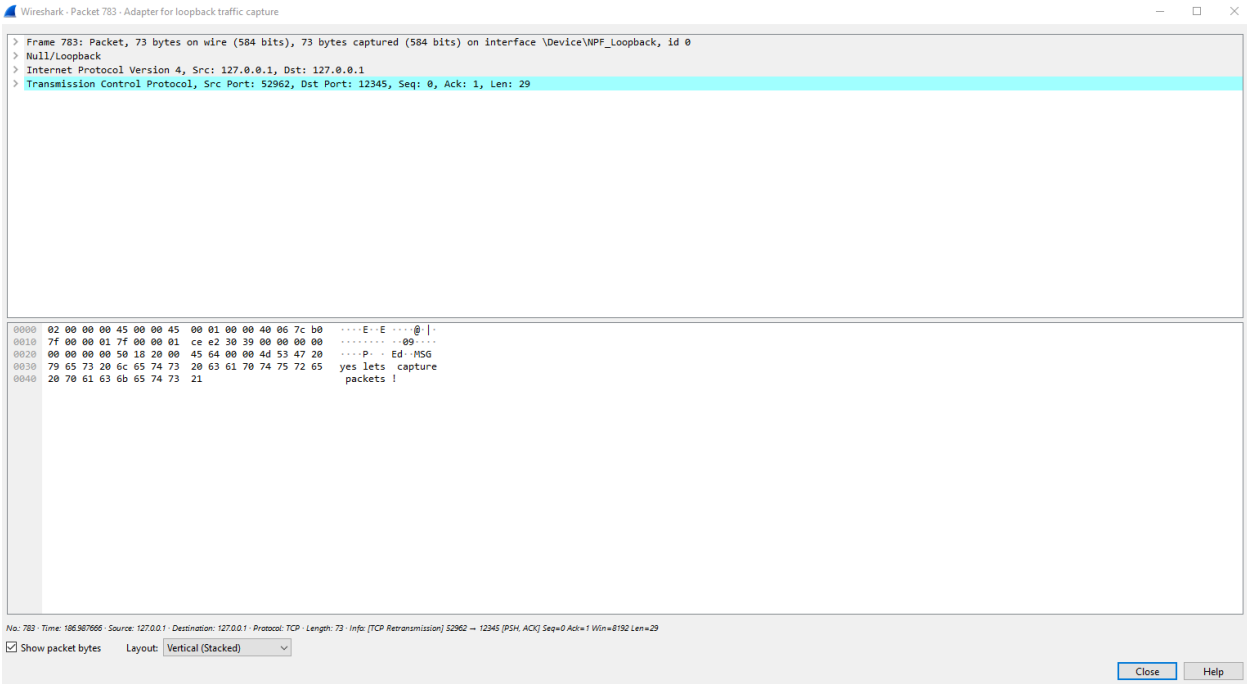
Example validation:

- CSV row: CHAT,50001,12345,HELLO alice,0.010
- Wireshark: packet shows TCP payload containing HELLO alice

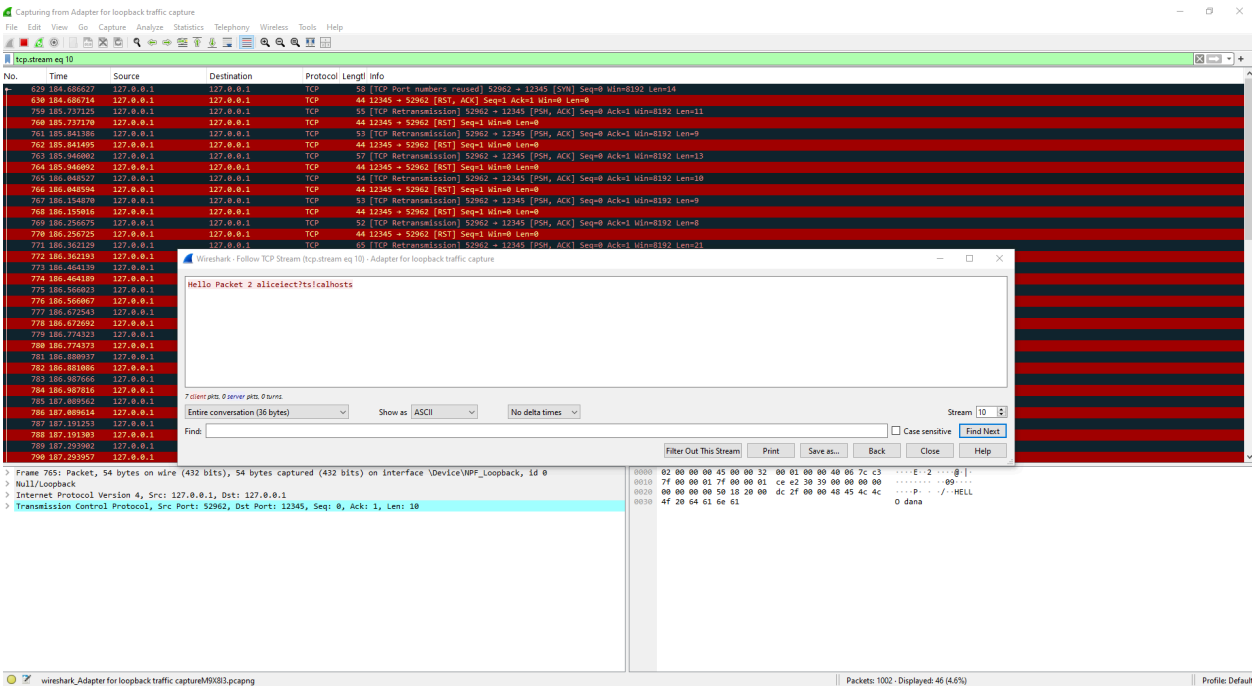
- Screenshot 1: Filter + packet list



- Screenshot 2:TCP ports and headers




- Screenshot 3: Payload showing the CSV message



# Part 2: Network Application and Traffic Analysis

## 1) General explanation of the system and code structure

For Part 2, I implemented a TCP client–server chat system using Python asyncio TCP streams (socket-based).

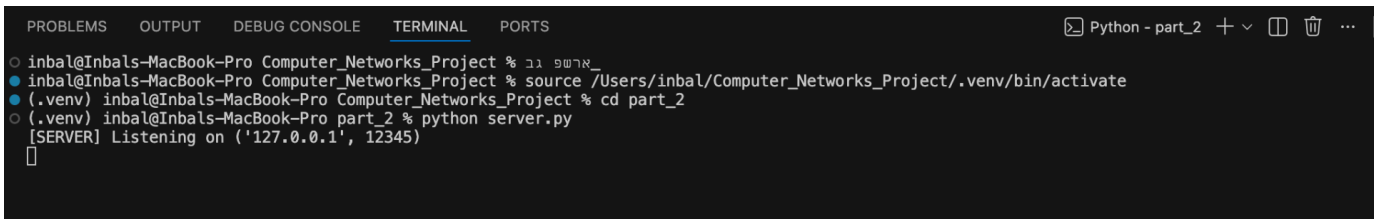
- **Main behavior:**
  - Clients connect to the server over TCP.
  - Each client identifies with a unique username using: *HELLO <username>*
  - The server supports private 1-to-1 chats (not group chat) using: *CHAT <username>*
  - Messages are exchanged using: *MSG <text>*
  - Clients disconnect using: *BYE*
  - The server supports at least 5 concurrent clients.

- **Data structures used:**
  - clients (dict): maps username → (reader, writer) for fast lookup and message routing.
  - pairs (dict): maps username → partner_username to track active private chats and quickly check if a user is busy.

- **The server handles edge cases such as:**
  - Unexpected client disconnects
  - Invalid or unknown commands
  - Username already taken
  - User not found
  - User already in an active chat

## 2) Installation and run instructions

1. Open the project folder in VS Code.
2. Make sure Python 3 is installed.
3. Start the server:
   *python server.py*



4. In a new terminal, start a client (repeat for multiple clients)
   *python client.py*

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   Python - part_2
inbal@Inbals-MacBook-Pro Computer_Networks_Project % cd part_2
inbal@Inbals-MacBook-Pro part_2 % source /Users/inbal/Computer_Networks_Project/.venv/bin/activate
(.venv) inbal@Inbals-MacBook-Pro part_2 % python client.py
[CLIENT] Connecting to 127.0.0.1:12345 ...
[SERVER] INFO Welcome! Please identify: HELLO <username>
```

5. Use commands in the client:
   *HELLO <username>, LIST, CHAT <username>, MSG <text>, BYE.*

## 3) Input and output examples

Below are examples of valid client input and expected output behavior.

Example 1 - Private chat between two clients

Client A:

```
(.venv) inbal@Inbals-MacBook-Pro part_2 % python client.py
[CLIENT] Connecting to 127.0.0.1:12345 ...
[SERVER] INFO Welcome! Please identify: HELLO <username>
HELLO John
[SERVER] OK HELLO
[SERVER] INFO Commands: LIST | CHAT <user> | MSG <text> | BYE
CHAT Paul
[SERVER] OK CONNECTED Paul
MSG hi Paul!
[SERVER] OK SENT
```

Client B:

```
(.venv) inbal@Inbals-MacBook-Pro part_2 % python client.py
[CLIENT] Connecting to 127.0.0.1:12345 ...
[SERVER] INFO Welcome! Please identify: HELLO <username>
HELLO Paul
[SERVER] OK HELLO
[SERVER] INFO Commands: LIST | CHAT <user> | MSG <text> | BYE
[SERVER] OK CONNECTED John
[SERVER] FROM John hi Paul!
```

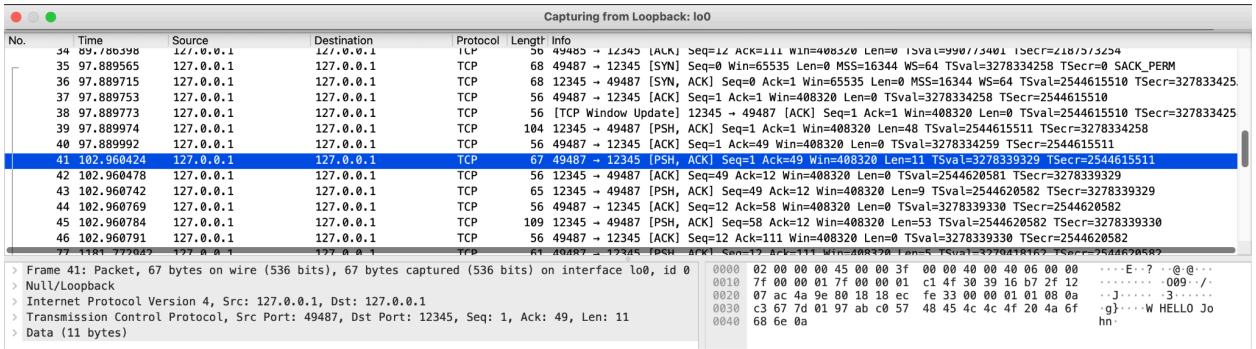
Client C:

```
(.venv) inbal@Inbals-MacBook-Pro part_2 % python client.py
[CLIENT] Connecting to 127.0.0.1:12345 ...
[SERVER] INFO Welcome! Please identify: HELLO <username>
HELLO George
[SERVER] OK HELLO
[SERVER] INFO Commands: LIST | CHAT <user> | MSG <text> | BYE
LIST
[SERVER] OK USERS George, John, Paul
```

Expected result:

- The server connects alice and bob privately
- Messages are delivered only between them

Example 2 - Disconnect



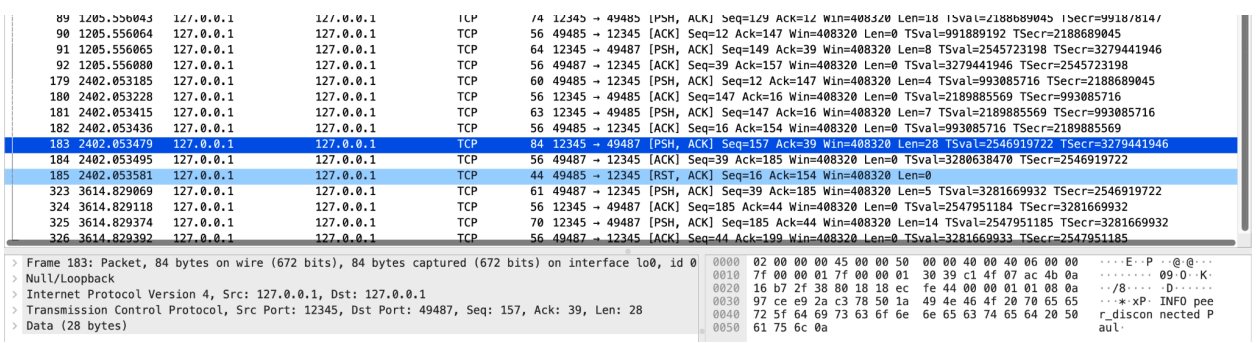

Client:

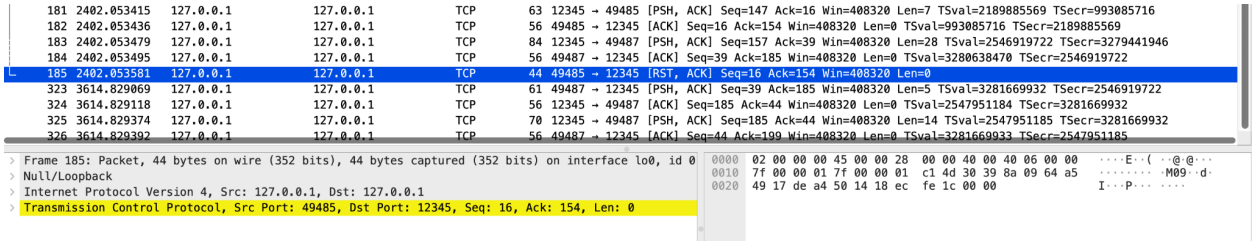
```
[SERVER] INFO Commands: LIST | CHAT <user> | MSG <text> | BYE
[SERVER] OK CONNECTED John
[SERVER] FROM John hi Paul
BYE
(.venv) inbal@Inbals-MacBook-Pro part_2 %
```

```
[SERVER] INFO Commands: LIST | CHAT <user> | MSG <text> | BYE
LIST
[SERVER] OK USERS John, Paul
CHAT Paul
[SERVER] OK CONNECTED Paul
MSG hi Paul
[SERVER] OK SENT
[SERVER] INFO peer_disconnected Paul
LIST
[SERVER] OK USERS John
```

Expected result:

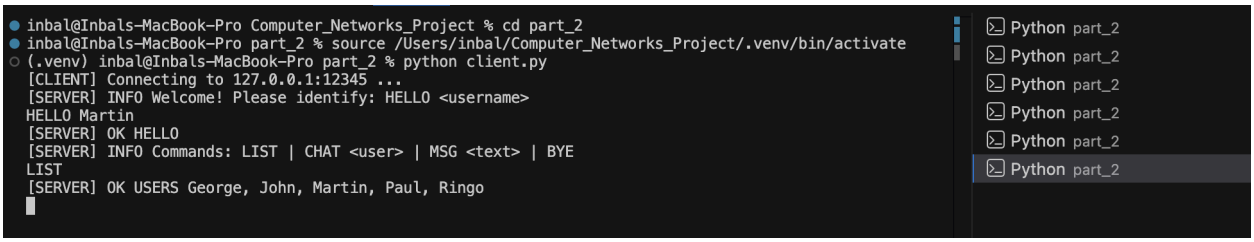
- The server closes the connection and removes the client from the active list





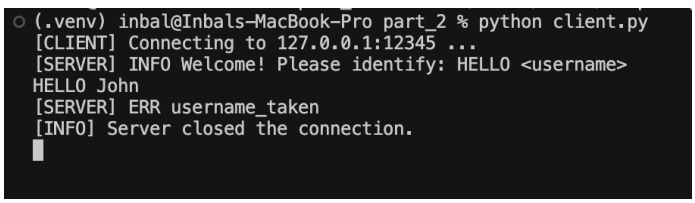

## - Concurrency and edge cases:

1. 5 clients connected simultaneously (John, Paul, George, Ringo, Martin)

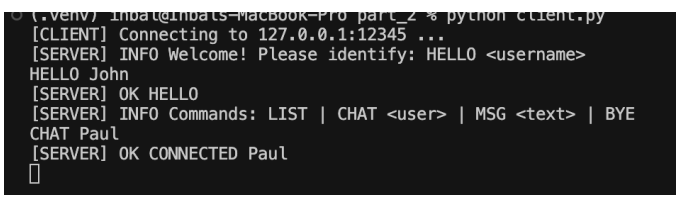


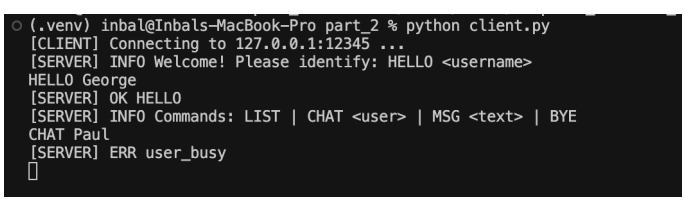

2. Edge case examples:

ERR username_taken: second client tried to use an existing username

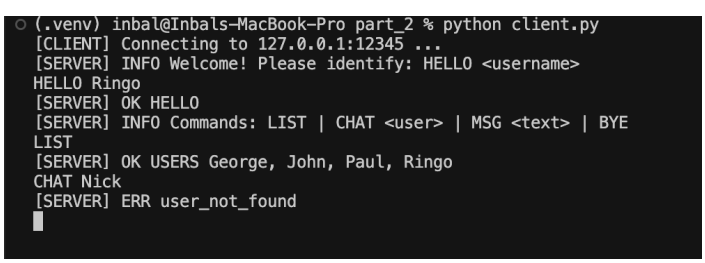



ERR user_busy: target user is already in an active private chat




ERR user_not_found: target username is not connected to the server



## 4) Traffic analysis of the application up to the Network layer

While running the chat application, I captured the traffic in Wireshark and analyzed it up to the Network layer (IP).

Observed elements in the capture:

- TCP connection establishment (3-way handshake: SYN → SYN/ACK → ACK)
- TCP data packets containing the chat protocol messages (HELLO, CHAT, MSG)
- TCP connection termination (FIN/ACK or connection close)
- IP header fields
    - Source IP
    - Destination IP
    - Protocol = TCP

- Screenshot 1: TCP handshake

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 21 | 63.689915 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 49695 → 12345 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=2121654819 TSecr=0 SACK_PERM |
| 22 | 63.690059 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 12345 → 49695 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=2260207512 TSecr=212165481 |
| 23 | 63.690093 | 127.0.0.1 | 127.0.0.1 | TCP | 56 | 49695 → 12345 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2121654820 TSecr=2260207512 |
| 24 | 63.690112 | 127.0.0.1 | 127.0.0.1 | TCP | 56 | [TCP Window Update] 12345 → 49695 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2260207512 TSecr=212165482 |

- Screenshot 2: TCP payload containing chat commands

```
INFO Welcome! Please identify: HELLO <username>

HELLO Paul

OK HELLO
INFO Commands: LIST | CHAT <user> | MSG <text> | BYE

;IST

ERR unknown_command

LIST

OK USERS John, Paul
OK CONNECTED George
```

- Screenshot 3: IP layer details (source/destination IP)

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 37 | 83.123636 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 49699 → 12345 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=3709852517 TSecr=0 SACK_PERM |
| 38 | 83.123783 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 12345 → 49699 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=1153272970 TSecr=370985251… |
| 39 | 83.123822 | 127.0.0.1 | 127.0.0.1 | TCP | 56 | 49699 → 12345 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=3709852517 TSecr=1153272970 |
| 40 | 83.123841 | 127.0.0.1 | 127.0.0.1 | TCP | 56 | [TCP Window Update] 12345 → 49699 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=1153272970 TSecr=3709852517 |
| 41 | 83.124042 | 127.0.0.1 | 127.0.0.1 | TCP | 104 | 12345 → 49699 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=48 TSval=1153272970 TSecr=3709852517 |
| 42 | 83.124079 | 127.0.0.1 | 127.0.0.1 | TCP | 56 | 49699 → 12345 [ACK] Seq=1 Ack=49 Win=408320 Len=0 TSval=3709852517 TSecr=1153272970 |
| 43 | 87.856574 | 127.0.0.1 | 127.0.0.1 | TCP | 67 | 49699 → 12345 [PSH, ACK] Seq=1 Ack=49 Win=408320 Len=11 TSval=3709857250 TSecr=1153272970 |
| 44 | 87.856628 | 127.0.0.1 | 127.0.0.1 | TCP | 56 | 12345 → 49699 [ACK] Seq=49 Ack=12 Win=408320 Len=0 TSval=1153277703 TSecr=3709857250 |

## 5) Use of AI

AI (ChatGPT + Claude) was used in this project.

## 6) Purpose of AI usage

ChatGPT and Claude were used to:

- Generate the CSV file with realistic application-layer messages
- Assist with designing the chat protocol commands and message flow
- Support writing and structuring the solution and report text

## 7) Example prompts

Examples of prompts used:

- “Generate a CSV file with chat messages including ports and timestamps”
- “Help me build a TCP client-server chat app in Python that supports private 1-to-1 chat”
- “How can I verify TCP payload messages in Wireshark?”